ESISA

Notes de cours Python

M. Lahmer

1. Syntaxe de base 4
1. Variables, assignation 4
2. Input / Output 4
3. Les opérateurs 5
4. Les blocs et les commentaires 5
2. Structures de contrôle 5
1. match/case 6
2. les boucle for et while 6
3. break, continue 6
3. Les chaines de caractères 7
1. Les fonctions de base 7
2. L'indexation et parcours 8
4. Listes, tuples, ensembles et dictionnaires 9
Les listes 9
Les tuples 9
Les ensembles 10
Les dictionnaires 10
5. Les Fonctions 10
Préliminaires 10
Portée d'une variable 11
Fonction Lambda 11
Map, Filter, Reduce 11
6. Gestion des exceptions 12
7. Les fichiers 12
8. Scripting avec Python 13
Gestion des fichiers 13
Execution des commandes shell 14
Les processus sous python 14
Instructions de Mise en attente 14
Les pipes 15
La redirection 16

8. Ploting avec python 16

9. Le package re 18

10. Base de données avec python 19

1. Syntaxe de base

C'est un langage interprété, objet mais il peut être utilisé en tant que langage de script

Un commentaire Python commence toujours par le caractère spécial **#**

#/usr/bin/python

Les différents types des variables sont donnés par :

- Les données numériques sont de type **int**, **long** ou **float** suivant la valeur que vous utiliser
- Les chaînes de caractères délimitées par " " ou ' ' ou """ """ sont de type **str**
- Les booléens qui vaudront **True** ou **False**

1. Variables, assignation

Python est un langage dynamiquement typé, les variables n'ont pas besoin d'être déclarées, et leur type peut changer au cours de l'exécution. la fonction type() permet de savoir le type d'une variable

```
>>>a = 3
>>>type(a)
<type 'int'>
>>>a = '3'
>>>type(a)
<type 'str'>
```

```
>>>x=3.4
>>>type(x)
<type 'float'>
>>>b=True
>>>type(b)
<type 'bool'>
```

```
>>>3443256367048874627473999435732 75**20 #** est l operateur puissance
548765985713255185942290206520374078101340449717842023651028169064607680
107554474445371776036514859467021485891219287396475322691986464615943485
746319235702156543512417751028955191028353050730398385128281188168428548
313235064941711955665503128619420718343189847749127636942027003483659900
576445642429166895066210907431314911982175481574234936567742865328090167
387145853465614042614379568436182532558460975757139559987167497637040786
080670976674911474013887733908973438625586445052678091464447319360087003
256890585995084147558595745904464179297934738930966154546077383011735473
752937847942595040121432078809696341558088712199605652131140232086181640
625L
```

La taille des entiers n'est limitée que par la mémoire de la machine

2. Input / Output

Affichage

```
print ("Demo x!=", 6)
```

on peut utiliser un affichage formaté en utilisant les formats classiques (int:%d),(float:%f),(bool et str:%s)

```
print ("My name is %s and weight is %d kg!" %'Mohamed', 76)
```

ou en utilisant la fonction format :'{0} {1}..{n}'.format(v0,v1,..vn)

```
print ("My name is {0} and weight is {1} kg!".format('Mohamed', 76))
```

Comme on peut utiliser le f-string comme suit

```
>>a,b=4,2
>>print(f'{a} est le carre de {b}')
```

Lecture

```
>>>X=input( "Donner une valeur : " )
```

Il faut saisir un type prédéfini python (int, float, string, bool,…)

3. Les opérateurs

les opérateurs arithmétiques sont (+, -, *, /, // (quotient), % (reste de division) et ** (puissance)). L'ordre de calcul d'une expression arithmétique tient compte de la priorité de chaque opérateur

(+,-) (*, /, //) (**)

```
>>> 5 - 8 + 4 * 2 ** 3
29
```

(5-8) + (4 * (2 ** 3))

les opérateurs logiques sont (==,!=, <,<=, >, >=, and, or, not). L'opérateur **in** permet de tester si une valeur appartient à une collection (chaîne de caractères, liste).

```
>>> s = 'SALAM'
>>> 'A' in s
True
```

Les opérateurs binaires permettent d'effectuer des opérations bas niveau en binaire sur des entiers

>>	décalage à droite	7 >> 1 vaux 3	111 >> 2 = 011
<<	décalage à gauche	8 << 2 vaux 32	1000 << 2 = 100000
&	le et binaire	5 & 3 vaux 1	101 & 011 = 001
^	le xor	5 ^ 3 vaux 6	101 ^ 011 = 110
|	le ou binaire	5 | 3 vaux 7	101 | 011 = 111
~	le complément à 1	~ 3 vaux -4 (-3 -1)	~ 011= - 100

4. Les blocs et les commentaires

L'indentation en python a une valeur syntaxique : elle sert à délimiter les blocs. Toutes les lignes d'un même bloc doivent être précédées du même nombre d'espaces blancs. Les retours à la ligne sont interdits en Python, sauf entre parenthèses (), crochets [], et accolades {}.

```
if (a == b
    and c == d):  print yes'
```

Un commentaire sur une seule ligne est précédé par **#**

Un commentaire sur plusieurs lignes ''' ligne commentaire 1
ligne commentaire 2 '''

2. Structures de contrôle

La seule construction conditionnelle existante en Python est **if… elif… else….** Toutes les branches sont optionnelles, à l'exception du if, il peut y avoir un nombre quelconque de elif, mais un seul else à la fin.

```
if a == b == c:
    print ('égaux')
elif a <= b <= c  or  c <= b <= a:
    print ('b au milieu')
elif b <= a <= c  or  c <= a <= b:
    print ('a au milieu')
else:
    print ('c au milieu')
```

1. match/case

python 3.10 a introduit match/case (switch avance base sur les patterns)

```
match value:
  case pattern1:
    # code
  case pattern2:
    # code
  case _:
    # default
```

Value peut être : types, sequences, dictionaries, objects or conditions

```
match day:
      case "sat" | "sun":
            print("Weekend")
      case "mon" | "tue" | "wed" | "thu" | "fri":
            print("Weekday")
```

2. les boucle for et while

Il existe deux types de boucles en Python. La plus couramment utilisée est le for... in qui permet de parcourir les éléments d'une liste.

```
for i in range(10):
    print (i)
```

La boucle for est souvent utilisée en conjonction avec la fonction **range**, dont la syntaxe générale est : **range(start, end, step) a**insi appelée, la fonction génère la liste des entiers entre start (inclus) et end (non inclus) avec pas de step :

```
>>range(0, 10, 2)
[0, 2, 4, 6, 8]
```

Les deux autres syntaxes admissibles sont range(start, end) (pas égal à 1) et range(end) (début égal à 0).

La deuxième boucle est très similaire à la boucle while en C.

```
a = 0
while a < 10 :
    a = a + 1
    print (a)
```

3. break, continue

Comme en C, l'instruction break sort de la boucle sans vérifier la condition :

```
for i in range(10):
    if i > 2 :
        break
    print (i)
0 1 2
```

```
for i in range(10):
    if i % 2 == 0:
        continue
    print (i)
1 3 5 7 9
```

L'instruction continue passe à l'itération suivante en sautant le reste du corps.

3. Les chaines de caractères

1. Les fonctions de base

la taille d'une chaîne de caractère n'est limitée que par la taille de la mémoire physique et l'architecture (32 ou 64 bits) de votre machine. Les chaînes de caractères délimitées par " " ou ' ' ou """ """ sont de type **str**

```
>>>str1 = "this is string example...."
>>>str2 = "exam";
La longueur est donnée par len
>>>len(str2)
4
La concaténation se fait par +
>>>str1+str2
this is string example...exam
La répétition
>>>str2*3
examexamexam
```

fonction	rôle	exemple
strip() lstrip() et rstrip()	élimine les espaces à gauche et à droite à gauche resp? à droite	' salam '.strip()—>'salam'
find (motif)	cherche motif dans une chaine et revoie sa position sinon -1	'dev python'.find('py') —> 4
startswith(prefix)	True si la chaine commence par prefix	'salam'.startswith('z')—> False
endswith(suffixe)	True si la chaine se termine par suffixe	'*.py'.endswith('py') —> True
replace(substr,anotherstr)	remplace substrat par anathoerstr dans une chaine	'*.py'.replace('py','backup') '*.backup'
isdigit() isalpha() isalnum() isspace()	permet de tester si une chaine est numérique, alphabétique, alphanumérique ou c'est des espaces	'1234'.isdigit() —> True 'A111'.isalnum() —> True 'AAAA'.isalpha() —> True 'A1 AA'.isspace() —> False

upper()
lower()
capitalize()
	Permet de transformer en Majuscule respect. en Minuscule
	Met le premier caractère en MAJ
		`'Salam'.upper() –> 'SALAM'`
		`'Salam'.lower() –> 'salam'`
		`'salut'.capitalize()->'Salut'`

count(sub)	retourne la fréquence de la sous chaine sub dans une chaine	`'salam'.count('a')–>2`
str (objet)	converti l'objet en une chaine	`str(13.0) –> '13.0'`
split(separateur)	Permet de décomposer la chaine dans un tableaux selon le séparateur (par défaut l'espace)	`'ceci est une demo'.split() –> ['ceci','est','une','demo']` `'login:pass'.split(':') –> ['login','pass']`

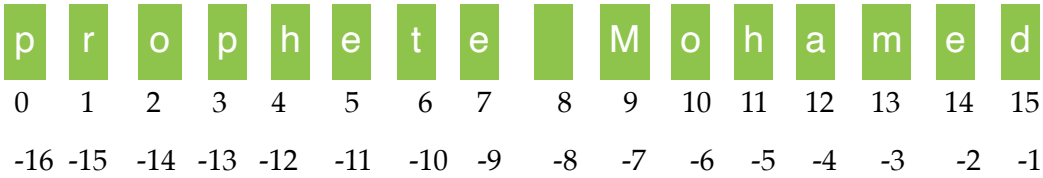
A noter que une chaîne de caractère sous python est non modifiable et n'accepte pas l'affection indexée : s='ESISA', s[0] = 'A' n'est pas autorisé

2. L'indexation et parcours

L'indexation

Pour indexer une chaîne on utilise l'opérateur [] dans lequel l'index est un entier qui peut être négatif.
En tant que programmeur, je vous déconseille d'utiliser les indexes négatifs c'est du n'importe quoi. Ainsi au lieu de travailler avec -i vous pouvez travailler avec len(s)-i.

s='prophete Mohamed'



Dans cet exemple len(s)=16. s[0] vaux 'p', s[8] vaux ' ' et s[-6]=s[len(s)-6]=s[10] vaux 'o'.
Attention une chaîne est un objet non mutable :on ne peut pas faire l'affectation par les indexes s[0]='Z' est interdite
Pour l'extraction d'une sous chaîne on utilise trois indexes [d:f:p] ou d,f est p sont respectivement début, fin et pas d'extraction (par défaut d=0,f=16 et p=1)

```
>>>s[9:16:1]
'Mohamed'
```

```
>>>s[::]
'prophete Mohamed'
```

```
>>>s[::3]
'pptMad'
```

Pour inverser un chaîne, en commence par la fin (ici 15) en utilisant un pas égale à -1.

```
>>>s[15::-1]
'demahoM etehporp'
```

Le parcours

Il existe deux manières pour parcourir une chaine

```
for c in s: print( c)
```

```
for index in range(len(s)): print(s[index])
```

```
L=[]
for i in range(1,101):
      if i % 2 == 0:
            L+=[i]
```

```
liste des diviseur de n
[d for d in range(1,n+1) if n % d ==0]
```

4.Listes, tuples, ensembles et dictionnaires

La longueur, l'indexation et le parcours sont identique à celui des chaînes de caractères

Les listes

L'un des objets les plus utilisés en Python, ce sont les listes. On déclare une liste avec les crochets [], et on accède à ses éléments de la même façon qu'on accède aux éléments d'une chaine

```
>>>L = [1, 2, 'a', True]
>>>L
[1, 2, 'a', True]
```

l'extraction se fait de la même manière que dans les chaines de caractères (voir 3.2)

```
>>> L[2::]
['a',True]
```

Pour initialiser une liste L=[] et l'ajout se fait de la même façon que les string en utilisant l'opérateur +. A titre d'exemple pour construire une liste contenant les nombres paires de 1 à 100

```
L = [ i for i in range(1,101) if i % 2 == 0 ]
```

Quelques méthodes prédéfinies

append(elt)	Ajoute un élément à la fine la liste	L=[] L.append(123)
index(elt)	renvoie l'indexe de la première occurence de let	L=[12,3,44,3,5,77] L.index(3) —->1
L.remove(elt)	supprime elt de la liste	L=[12,3,44,3,5,77] L.remove(3) —->L devient [12,44,3,5,77]
L.reverse()	inverse les places des éléments d'une liste	L=[True,[1,2]] L.reverse() —->L devient [[1,2],True]
L.sort()	permet de trier le contenu d'une liste. Attention les éléments de la liste doivent être naturellement comparable	L=["z", "d", "a"] L.sort() —> L devient ["a", "d", "z"]
len(L),min(L), max(L), sum(L)	longueur, min, max ou la somme des éléments d'une liste (même condition que ce qui précède)	L=[23,1,4,2] min(L)—>1 max(L) —->23 sum(L) —>30
L.pop()	retourne le dernier element de L et le retire (Dépiler)	L=[23,1,4,2] L.pop()—>2 et L deviendra [23,1,4]
L.insert(pos,element)	insert un element dans la position pos	L=[23,1,4,2] L.insert(0,5)—->L=[5,23,1,4,2]
L.clear()	initialise la liste	L=[23,1,4,2] L.clear()——> L=[]

Les tuples

Un tuple est une liste non modifiable c'est les types énumérés en java. On les utilise pour définir les jours de la semaines, le mode de payement, les couleurs,… On ne peut pas faire l'affectation directe

couleur=("red","green","blue")
couleur[1]—> "green"
len (couleur) —> 3
pour supprimer un tuple on utilise le mot clé del
del couleur

Les ensembles

Un set est une liste d'elements uniques non ordonnés. On peut construire un set à partira d'une liste.
S={1,2,3} ou T=set([2,2,3,6,7])
S & T donne l'intersection de S et T ici c'est l'ensemble {2,3}
S | T donne l'union de S et T {1,2,3,6,7}
S - T les éléments de S qui ne sont pas dans T {1}
S <= T retourne True si S est inclus dans T False sinon

Les dictionnaires

Un dictionnaire est une liste d'objets ou chaque objet est composé d'une clé et d'une valeur. La clé et la valeur sont concatenés par : sous la forme de k:v.

tele = {'directeur':112,'secretaire':113,'personnel':200}
tele['secretaire']—>113
tele.keys() —->['directeur','secretaire','personnel']
tele.values() —> [112,113,200]
Si on veut parcourir à la fois les clés et les valeurs :
for k,v in tele.items() :
    print (k,v)
pour ajouter un élément au dictionnaire tele['it']=115, pour ajouter plusieurs éléments
tels.update({'finance':120,'dev':117})

5. Les Fonctions

Préliminaires

Les fonctions Python sont définies par le mot clef **def**. Elles peuvent prendre un nombre arbitraire de paramètres, et renvoient une valeur à l'aide du mot clef return. Toute fonction renvoie une valeur, les fonctions qui n'ont pas de return renvoient la valeur spéciale None.

```
def max(x, y):
    if x > y:
        return x
    else:
        return y
```

Certains paramètres peuvent prendre des valeurs par défaut. Si un paramètre prend une valeur par défaut, tous ceux qui le suivent doivent aussi en prendre.

```
def pow(x, n=0):
      return x**n
>>>pow(3) # quand on ne donne pas n, par défaut n=0
1
>>>pow(3,2)
9
```

Une fonction sous python peut prendre un nombre d'arguments variable *args

```
def prod(*args):
    pr=1
    for p in args:
        pr*=p
    return pr
print (prod(2,3))   #Affichera 6
print (prod(2,3,4)) #Affichera 24
```

```
Il existe aussi les fonctions à paramètre clés (l'ordre n'est pas important) :
def process(**args):
     for k,v in args.items():
          print(k,v)
process(sys='win',version='7')
sys win
version 7
```

Portée d'une variable

une variable initialisée à l'intérieur d'une fonction est une variable locale à la fonction.

```
var=10
def demo() :
      var=3
print (var) # on aura toujours 10
```

Pour dire à python que var de demo est une variable globale il faut ajouter le mot clé global.

```
def demo() :
      global var
      var=3
print (var)# on aura 3
```

Fonction Lambda

Une fonction lambda est une fonction anonyme qui peut prendre n'importe quel nombre d'arguments, mais ne peut avoir qu'une seule expression.

lambda arguments : expression

Map, Filter, Reduce

Map, Filter et Reduce sont des paradigmes de la programmation fonctionnelle. elles permettent d'écrire un code plus simple et plus court, sans nécessairement avoir à se soucier des boucles et des branchements conditionnels. Essentiellement, ces trois fonctions vous permettent d'appliquer une fonction sur un certain des itérables (List, Dictionnaire, tuple, set…)

Map permet d'appliquer une fonction sur une collection d'objets

- syntaxe : map(function, iterable, …)
- numbers_1=[2,4,7]
- numbers_2=[-3,1-5]
- product=list(map(lambda x, y : x * y, numbers_1,numbers_2)
- print(product)

Filter permet d'appliquer un filtre sur une collection d'objets

- words = ("aya", "write", "madam", "laval", "esisa", "kiosk")
- palindromes = list(**filter**(lambda word: word == word[::-1], words))
- print(palindromes)

Reduce permet d'appliquer une fonction sur une collection d'objets pour en retourner un

- from functools import reduce
- numbers = [0, 1, 2, 3, 4]
- print(reduce(lambda x, y : x + y, numbers) #affiche 10
- ((((0 + 1) + 2) + 3) + 4)

```
try :
        print(x)
except :
        print('variable non definie')
finally :
        print('fin traitement')
```

6. Gestion des exceptions

Lorsqu'une erreur (exception) se produit, Python s'arrête normalement et génère un message d'erreur. Ces exceptions peuvent être gérées à l'aide de bloc **try/except.**

```
>>print(10/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

```
try :
        print(10/0)
except :
        print('division par zero impossible')
>>division par zero impossible
```

Vous pouvez définir autant de blocs d'exception que vous le souhaitez, par ex. si vous souhaitez exécuter un bloc de code spécial pour un type d'erreur particulier :

```
try:
        a=0
        print(b/0)
except NameError:
        print('variable indefinie')
except ZeroDivisionError:
        print('division par zero impossible')
```

Il est possible de générer votre propre exception en utilisant le mot clé **raise,** mais on affichera l'exception et on quitte

```
if a == 0 : raise Exception("Sorry, no numbers below zero")
```

7. Les fichiers

La fonction open(fichier,mode) permet d'ouvrir un fichier selon un mode (r, w, a, r+). Sous windows, il y a une distinction entre les fichiers binaires et texte ainsi pour les fichiers binaires en retrouve en plus les modes (rb, wb et r+b).

```
#!/usr/bin/python
# Open a file
fo = open("foo.txt", "wb")
print "Name of the file: ", fo.name
print "Closed or not : ", fo.closed
print "Opening mode : ", fo.mode
```

La fonction open retourne un objet qui dispose des méthodes pour l'écriture, la lecture, le positionnement et la fermeture.

On crée un fichier vide dans le répertoire courant

```
>>> file=open("demo.txt","w")
```

On écrit une deux lignes

```
>>> file.write("Ce ci est un test \n123456789")
```

On ferme le fichier puis on l'ouvre en mode lecture écriture
>>> file.close();
>>> file=open("demo.txt","r+")
Pour lire la totalité du fichier
>>> print (file.read())
Ce ci est un test
123456789
Pour revenir au début, on utilise la fonction seek(offset)
>>> file.seek(0)
On peut lire un nombre de caractères bien définit file.read(nb_char)
>>>file.read(5)
Ce ci
Pour lire une ligne du fichier, on utilise file.readline() sinon pour lire toutes les lignes on utilise la fonction file.readlines() (retourne une liste)
>>>for ligne in file.readlines():
print (ligne)
la fonction enumerate permet de renvoyer le numéro de ligne courante dans un fichier texte.
>>> for nl,ligne in enumerate(file):
... print (nl,ligne)
0 Ce ci est un test
1 123456789
Pour s'assurer de la fermeture d'un fichier il est recommandé d'utiliser

8. Scripting avec Python

Gestion des fichiers

os.mkdir("test") : permet de créer un nouveau répertoire test

os.chdir("test") : c'est cd, elle permet d'accéder au répertoire test

os.getcwd() : retourne le nom du répertoire courant

os.rmdir("/home/user/test") : supprime un répertoire (il doit être vide)

dirs = os.listdir(path) : retourne le contenu d'un répertoire sous forme de liste

os.remove("demo.txt") : supprime un fichier

os.rename(AncienNom,NouveauNom) : renomme un fichier

Les tests de base sur les dossiers et fichiers se font tous à partir de fonctions du module os.path.

os.path.exists(*path*) : Permet de vérifier si path est un chemin réel.

os.path.isabs(*path*) :Teste si path est un chemin absolu.

os.path.isdir(*path*) :Teste si path pointe vers un dossier.

os.path.isfile(*path*) :Teste si path pointe vers un fichier.

os.path.abspath(*path*) : Retourne le chemin absolu du chemin relatif path donné en argument.

os.path.basename(*path*) : Retourne le dernier élément du chemin.

os.path.dirname(*path*) : Retourne le chemin pour accéder au dernier élément.

os.path.getatime(*path*), os.path.getmtime(*path*)¶, os.path.getctime(*path*) : retourne respectivement le dernier temps d'accès, de modification et de changement d'état

os.path.getsize(*path*) : Retourne la taille d'un fichier en octets

Exercices :
script find

Ecrire un script en python permettant d'afficher de façon récursive les fichiers d'un répertoire dont le nom contient un motif.

Execution des commandes shell

Pour exécuter une commande sans traiter le résultat de retour on utilise la fonction system

os.system("clear")

Sinon on fait appelle à os.popen (commande) qui permet d'exécuter une commande dans un pipe que vous pouvez utiliser à travers une variable.

```
ll=os.popen('ls -l')
for ligne in ll:
 print ligne
ll.close()
```

Attention l'execution est asynchrone c.à.d on attend pas à ce que le processus se termine pour continuer. Désormais, Il existe le module subprocess beaucoup plus flexible :

```
import subprocess
#run vous permet d'exécuter une commande et récupérer son résultat
process  = subprocess.run('ls',capture_output=True, text=True)
print(process.output)
```

Pour plus de contrôle sur les entrées/sorties, vous avez Popen

```
process = subprocess.Popen(['ls','-l'],
                 stdout=subprocess.PIPE, stderr=subprocess.PIPE)
stdout, stderr = process.communicate()
print(stdout)
```

Pour avoir le code de retour de l'execution de la commande on utilise la méthode poll().

```
print(process.poll()) #0 si Ok
```

Les processus sous python

Un processus est un programme en cours d'execution. Un processus passe par plusieurs états avant qu'il se termine :

endormie (S), en exécution (R), suspendu (T) voir zombie (Z)

Pour avoir les caractéristiques de base d'un processus (comme ps linux)

os.getpid() : retourne le pid du processus en cours

os.getppid() : retourne le ppid

os.getuid() : retourne le uid de l'utilisateur propriétaire du processus

os.getgid() : retourne le gid du groupe propriétaire du processus

Pour créer un processus fils on utilise la fonction os.fork(). La fonction fork retourne trois valeurs (0 : le fils, >0 : le père, -1 erreur).

```
import os, sys
pid_fils=os.fork()
if pid_fils == -1 :
      print('erreur creation')
      sys.exit(0)
elif pid_fils == 0 :
      print('Je suis le Fils PID : %d, PPID : %d' %(os.getpid(),os.getppid()))
else :
      print('Je suis le Pere PID : %d, PPID : %d' %(os.gettpid(),os.getppid()))
```

Instructions de Mise en attente

On peut mettre un processus dans l'état endormis en utilisant :

- Une instruction E/S comme input
- Attente temporelle time.sleep(s) (time.sleep(0.1) attente 100 ms)

- Attente sur la terminaison d'un fils os.wait() (pid,status=os.wait() retourne le pid du fils qui a terminé et l'état de retour associée)
- Attente d'un signal signal.pause()

Pour forcer un père à attendre la fin de son fils en utilisant un signale ici SIGUSR1

```
import os, sys, signal
pid_fils=os.fork()
if pid_fils == -1 :
      print('erreur creation')
      sys.exit(0)
elif pid_fils == 0 :
      print('Je suis le Fils PID : %d, PPID : %d' %(os.getpid(),os.getppid()))
      kill(os.getppid(),signal.SIGUSR1)
else :
      signal.pause()
      print('Je suis le Pere PID : %d, PPID : %d' %(os.gettpid(),os.getppid()))
```

pour redéfinir le traitement du père une fois il reçoit le signale, on ajoute la fonction signal.signal(signal,handler)

```
def handler(num_sig, pile):
      print('Signal recu %d' %(num_sig))
def code_fils(pid):
    print('Traitement du fils : %d' %(pid))
def code_pere():
    print('Traitement du pere')
pid_fils=os.fork()
if pid_fils == -1 :
    print ('erreur creation processus')
    sys.exit(0)
if pid_fils == 0 :
     code_fils(os.getpid())
     os.kill(os.getppid(),signal.SIGUSR1)
else :
      signal.signal(signal.SIGUSR1,handler)
      signal.pause()
```

Les pipes

un pipe (tube) est une structure logique en mémoire avec deux extrémités une pour la lecture et autre pour l'écriture. C'est un moyen de communication unidirectionnel entre processus. Le système conserve temporairement l'information transmise (écrite) jusqu'à ce qu'elle soit lue par le processus de réception.

Pour la communication bidirectionnelle entre les processus, deux pipes peuvent être mis en place, un pour chaque direction. Les processus utilisant des canaux doivent avoir un processus parent commun.

```
import os,sys,signal
r,w = os.pipe()
pid_child = os.fork()
if pid_child == 0 :
```

```
    #Tache du fils
    fp=os.popen('ls -l','r')
    result = fp.read()
    fp.close()
    #Ecriture du resultat
    os.close(r)
    fw = os.fdopen(w,'w')
    fw.write(result)
    fw.close()
    #Fin du fils
    sys.exit(10)
    elif pid_child >0 :
    #Attente la terminaison du fils
    pid,status=os.wait()
    os.close(w)
    #Lire le resultat depuis le pipe
    fr=os.fdopen(r,'r')
    result=fr.read()
    print('Message recu :',result)
    fr.close()
```

La redirection

Chaque processus reconnaît trois descripteurs de fichiers standards à savoir :

- stdin (0) : l'entrée clavier
- stdout (1) : l'écran
- stderr (2) : sortie d'erreur par défaut l'écran

Pour dire à un processus d'écrire dans un fichier au lieu de l'écran ou bien de lire depuis un fichier au lieu du clavier on fait appel aux fonctions systèmes dup et dup2 :

- os.dup2(newfd,oldfd) : permet de remplacer l'ancien descripteur oldfd par newfd

Exemple :

On veut mettre en place un script qui réalise ls -l | grep '.py'. On aura besoin de deux processus (père et fils) et un pipe. Le processus fils exécutera la commande ls -l et au lieu d'écrire le résultat à l'écran il les redirige vers le pipe. Le père (grep '.py') va lire son entrée depuis le pipe au lieu du clavier. Pour exécuter une commande, on va faire appel à os.execvp().

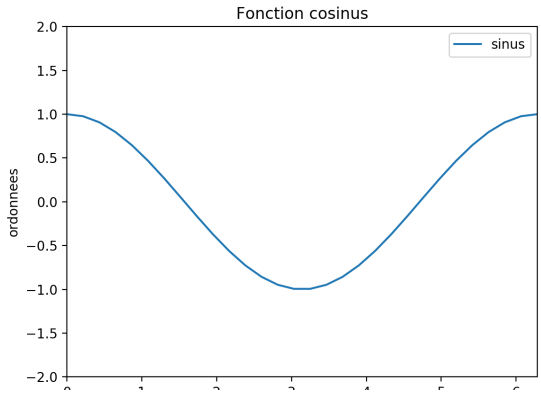
```
import os,sys,signal
r,w = os.pipe()
pid_child = os.fork()
if pid_child == 0 :
    os.close(r)
    os.dup2(w,1)
    os.execvp('ls',['ls','-l'])
elif pid_child >0 :
    #Attente la terminaison du fils
    pid,status=os.wait()
    os.close(w)
    os.dup2(r,0)
    os.execvp('grep',['grep','.py'])
```

8. Ploting avec python

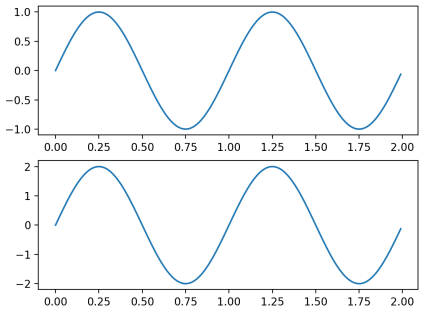
Il faut installer les bibliothèques matplotlib et numpy (les calcules) :

```
sudo apt install python3-pip
python -m pip install -U pip
python -m pip install -U matplotlib
```

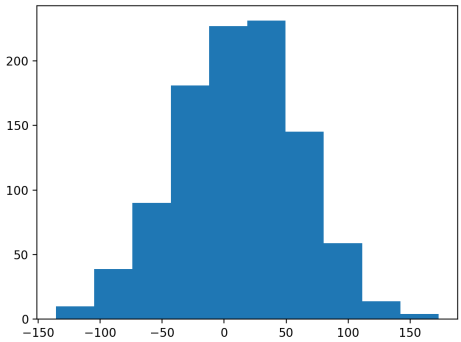
```
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 2*np.pi, 30)
y = np.cos(x)
#definition des limites des axes
plt.xlim(0, 2*np.pi)
plt.ylim(-2, 2)
#Titre de la figure
plt.title("Fonction cosinus")
plt.plot(x, y,label="cosinus")
#afficher la légende
plt.legend()
#nom des axes
plt.xlabel("abscisses")
plt.ylabel("ordonnees")
plt.show() # affiche la figure a l'ecran
```




```
On peut dessiner plusieurs courbes dans la
même figure pour cela on utilise subplot
import matplotlib.pyplot as plt
import numpy as np
t = np.arange(0.0, 2.0, 0.01)
s1 = np.sin(2*np.pi*t)
s2 = np.sin(4*np.pi*t)
plt.figure(1)
'''2 le nombre de lignes, 1 le nombre
de colonnes et 1 l'indexe de la figure
'''
plt.subplot(211)
plt.plot(t, s1)
plt.subplot(212)
plt.plot(t, 2*s1)
plt.show()
```

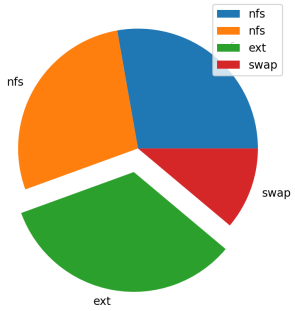



```
Pour dessiner un histogramme
from matplotlib import pyplot as plot
import numpy as np

# variable gaussienne
data = np.random.randn(1000)
# plot normed histogram
plot.hist(data)
plot.show()
```




```
import matplotlib.pyplot as plt
```

```
import numpy as np
y = np.array([25, 25,30, 10])
mylabels = ["ntfs", "ntfs", "ext", "empty"]
myexplode = [0.2, 0, 0, 0]

plt.pie(y, labels = mylabels, explode =
myexplode, shadow = True)
#l'attribut autopct='%1.1f%%' vous permet
#d'écrire les % sur les tranches
plt.legend()
plt.show()
```



9. Le package re

Le module re est prédéfini dans Python et il dispose de fonctions qui permet d'effectuer une recherche dans un texte en se basant sur la notion d'expression régulière.

. : un caractère quelconque
(expr)? : expr 0 ou une fois
(expr)+ : expr apparaît une ou plusieurs fois
(expr)* : expr apparaît 0 ou plusieurs fois
(expr)**{n}** : expr apparaît exactement n fois, {n, m} entre n et m
^expr : débute par expr
expr**$** : se termine par expr
(expr1 | expr1) : expr1 ou expr2
[a-z] :intervalle des caractères de a à z
[1570] : 1 ou 5 ou 7 ou 0
[^abc] : tout caractère à l'exception de a, b et c
[15][29] : tout nombre composé de deux chiffres de 12 à 59
\w : Lowercase w. Matches any single letter, digit, or underscore.
\s : Lowercase s. Matches a single whitespace character like: space, newline, tab, return.
\d : Lowercase d. Matches decimal digit 0-9.

-re.compile(*pattern, flags=0*) : compile l'expression régulière en un objet sur lequel on peut appliquer les méthodes match ou search. Il existe plusieurs type de flag tel que par exemple re.I qui permet d'ignorer la case.

```
prog = re.compile(pattern)
result = prog.match(string)
s='esisa ecole Ing depuis 1998'
prog=re.compile('es') #chercher la premiere occurrence de es
prog.match(s)==True #affichera True
prog=re.compile('ez')
prog.match(s)==True #affichera False
```

Le package re dispose de 3 méthodes supplémentaire importantes :

re.search(pattern, string) : return True si pattern est dans s False sinon mais il est possible d'avoir la première occurrence de pattern comme suit : re.search(pattern, string).group()
re.findall(pattern, string) : retourne une liste contenant toutes les occurrences de pattern dans string
re.findall('[0-9]',s) —-> ['1', '9', '9', '8']
re.sub(pattern, replace, string) : retourne une nouvelle chaine dans laquelle pattern est remplacé par la chaîne replace.
Dans cet exemple on va remplacer un ou plusieurs espace par ;
news=re.sub('\s+',';',s)——>news='esisa;ecole;Ing;depuis;1998'

10. Base de données avec python

Premièrement il faut installer le connecteur, chaque sgbd dispose de son propre connecteur. Dans le cas de mysql il faut faire : **pip install mysql-connector-python**
1- Pour se connecter à une base de données

```
import mysql.connector as mysql
Con = mysql.connect(
         host = "localhost",
         user = "root",
      passwd = "dbms"
      database = "dbname"
     )
2- Créer un objet cursor qui sera utilisé pour exécuter une requête SQL
cursor = con.cursor()
```

3- Exécuter votre ordre Sql, à ce niveau on distingue plusieurs types :
a- Ordres de définition des données (create , alter, drop…)

```
cursor.execute("CREATE TABLE users (id INT(11) NOT NULL AUTO_INCREMENT PRIMARY KEY, display_name VARCHAR(255), user_name VARCHAR(255)) ")
```

b- Ordres de manipulation ou de selection (select, show, desc,…)

```
cursor.execute("SHOW TABLES")
tables = cursor.fetchall()
for table in tables:
   print(table)
```

c- Ordre de manipulation (insert, update, delete)

```
#Exemple d'insertion pour se protégé contre l'attaque SQL Injection
query = "INSERT INTO users (name, user_name) VALUES (%s, %s)"
#Les valeur à ajouter sont données sous forme de tuple
values = ("Esisa", "Ecole Supérieur …… ")
#Execution de la requête
cursor.execute(query, values)
#Pour finaliser
con.commit()
# print(cursor.rowcount, 'Enregistrement inseres')
```

```
Vous pouvez exécuter plusieurs ordre en une seule instruction avec
cursor.executemany(Query,listOftuplesValues)
data = [
  ('Jane', date(2005, 2, 12)),
```

```
  ('Joe', date(2006, 5, 23)),
  ('John', date(2010, 10, 3)),
]
stmt = "INSERT INTO employees (first_name, date) VALUES (%s, %s)"
cursor.executemany(stmt, data)
```

Notez bien que par mesure de sécurité il faut traiter les transactions sql dans un bloc try except. En effet la connection à une base de données peut échouer pour plusieurs raisons (host inexistant, login ou mot de passe incorrecte , base de données introuvable). Ci-dessous des exemples de fonctions utilisant try except :

```
def existsDataBase(name):
    try :
        cursor.execute('SHOW DATABASES')
        databases=cursor.fetchall()
        return (name,) in databases
    except Exception as e:
        print(e)
def createDatabase(name):
    try :
        if not existsDataBase(name):
            cursor.execute('CREATE DATABASE '+name)
        cursor.execute('use '+name)
    except Exception as e:
        print(e)
def createTable(name):
    try:
        cursor.execute('CREATE TABLE IF NOT EXISTS '+name+''' (id int primary
key auto_increment,
        name varchar(30),password varchar(30))''')

    except Exception as e:
        print(e)
def insert(user):
    try:
        cursor.execute('INSERT INTO user (name,password) values(%s,%s)',user)
        cnx.commit()
    except Exception as e:
        cnx.rollback()
```